Formal Verification of a Realistic Binary Rewriter

Abstract. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

1 Introduction

Development of mission-critical software systems, such as secure OSes, VMs, hypervisors, and runtime systems intended for deployment in adversarial environments, demands exceptionally high-assurance development tools and procedures to meet strict safety, security, and correctness standards. Machine-checked formal verification has emerged as a hallmark approach for achieving the requisite guarantees for such systems, with success stories such as CompCert [1], seL4 [2], and CakeML [3] demonstrating the efficacy of such *formal methods* (FM) for obtaining real-world, high-assurance systems in practice. These successes rely on mature theorem proving environments such as Rocq [4] and Isabelle/HOL, which have enabled rigorous, machine-checked reasoning about source-level semantics and compiler correctness.

While machine-certified compilation has received extensive attention in the literature (cf., [5–11]), binaries that execute on real mission-critical systems are seldom the unaltered output of a compiler. Compiled binaries undergo a variety of low-level transformations prior to execution, including load-time linking, hardening, and sandboxing, which prepares the binary to safely execute in the context of the surrounding OS or VM. The *binary rewriters* that implement these transformations demand an equivalent or higher level of assurance, since any flaw could undermine all assurances achieved during compilation. However, complete formal verification of a binary rewriter *implementation* has remained a difficult challenge for FM, primarily because their correctness properties are universal quantifications over arbitrary binary code. In particular, a binary rewriter

is correct if and only if it achieves a desired safety property for all possible input binaries, not merely those produced by a particular compiler or that limit themselves to a particular subset of the ISA.

Binary *control-flow integrity* (CFI) is a well-studied example of the associated challenges [12–19]. Source-agnostic CFI algorithms transform input binaries so as to confine all their reachable control-flows to a policy-specified whitelist of edges. Implementations typically identify and instrument the target binary's control-flow transfer instructions (e.g., jumps, calls, and returns) with extra *guard code* that checks each impending destination against the policy at runtime. Impending violations solicit a safe intervention, such as secure abort, rollback, or restart. A binary CFI implementation is *sound* if all reachable execution traces of all binaries it generates obey the policy.

Since source-driven (*top-down*) FM approaches cannot state or verify a source-agnostic soundness property, and since binary-driven (*bottom-up*) FM approaches are typically designed to verify concrete input binaries (not the arbitrary output of a binary transformation), prior efforts to verify binary CFI have resorted to verifying each CFI-hardened binary case-by-case (e.g., [**rocksalt**]). This helps code-consumers protect themselves from untrusted code-producers, but it does not empower code-producers to create a verifiably sound CFI implementation that always passes the consumer's checks. Producers must resort to unit testing (i.e., running the rewriter on many input binaries to see whether any resulting binaries fail verification), losing the complete-coverage benefits of FM and risking denial of service if the deployed rewriter fails.

Formally verifying a binary rewriter implementation requires overcoming two major hurdles: First, it requires a verification framework that supports *underconstrained-code properties*—properties in which the target code is a quantified proof variable, not a concrete program. Such properties are inexpressible in most existing bottom-up FM frameworks (e.g., [20–24]). Second, the approach must have a capability for reasoning about arbitrary byte sequences as (possibly invalid) binary code. This is necessary because rewriter implementations read and write bytes, and their soundness is therefore contingent upon their correct interpretation of those bytes as instructions. Idealizing a rewriter as an abstract model of its algorithm waives verification of many of the most complex parts of its implementation, which are pernicious sources of potential error.

In this paper we address this long-standing challenge through the design and implementation of Picinæ, a platform in Rocq for instruction-level analysis of executables. Picinæ is designed from the ground up to construct machine-checked proofs about real binary programs for real ISAs, while maintaining sufficiently general foundations to prove theorems in which all or part of the target binary code is a proof variable rather than a specific program. It supports modeling both von Neumann and Harvard architectures, including difficult ISA features like misaligned instructions on Intel, conditional instruction prefixes on ARM, undefined and non-deterministic effects, memory access permissions, self-modifying code, hardware exceptions, indirect calls and returns, mixed endianness on ARM, instructions with internal microcode loops, and multi-ISA

integrated systems. These capabilities are leveraged to prove that a realistic binary CFI implementation for a mainstream ISA (ARM7) safely hardens arbitrary binaries (including rejecting unsafe or malformed binaries that it is designed to exclude from execution). Specifically, we present the following contributions:

- the Picinæ framework, which facilitates machine-checked proofs of correctness for binary programs and binary transformations;
- a dependently-typed, co-inductive foundation for proving correctness properties of algorithms that manipulate machine code; and
- a case study demonstrating the first machine-checked proof of soundness for a binary CFI rewriter implementation targeting ARM7 microprocessor code.

2 System Overview

[KWH Comment)] I don't like this figure. First, it looks aesthetically clunky. We should create something more professional-looking here (e.g., shadowed boxes, a theme-driven color palette, consistent font sizes, etc.). Second, the flow is confusing because it looks like the external lifter goes to the internal lifter goes directly to the proof. I understand why you're depicting it that way, but it's unexpected and jarring to a newcomer. We don't want anything jarring at this point in the paper. The external and internal lifters should have separate arrows that both lead into the Picinae IL. That should flow into the Symbolic Interpreter (not "executor"), not into Theory. Splitting between the two makes it look like they're alternatives to each other (which obviously isn't what we want to say). Symbolic interpretation should flow into ISA Model, but that should then flow into some kind of central box perhaps labeled "Proof Generation". That central box should have satellite boxes feeding into it with a variety of labels like: Binary Arithmetic Theory, (Co-)Inductive Theory of Traces, Memory Safety Theory, etc. We don't need to list everything Picinae has; we just need the ones to which we'd like to refer in this paper when explaining how things work. The final output from Proof Generation should be Proof.v, probably outside the Rocq system box entirely.

Figure 1 shows an overview of Picinæ's workflow. Verifying a concrete binary entails first automatically *lifting* it to a low-level, ISA-agnostic IL that explicitly encodes each instruction's machine-level effects. Lifting can be accomplished with external tools or as Rocq code that produces IL from machine code bytes. To support external lifters, we implement plugins for Ghidra [35] and BAP [25] that yield Picinæ IL from P-code and BIL code, respectively. To support internal lifting, we implement instruction decoders for RISC-V and ARM7 in Gallina.

Internal lifting is critical for verifying a binary rewriter, since it allows proofs to reason about instruction encodings. For example, a secure CFI implementation must not miss any control-flow transfer instructions in the target binary. This can be proved by verifying that its instruction decoding logic correctly identifies and guards all opcodes whose ISA semantics perform control-flow transfers.

Lifting does not attempt perfect disassembly since it does not attempt to recover any control-flows from the binary. It merely decodes the bytes at every

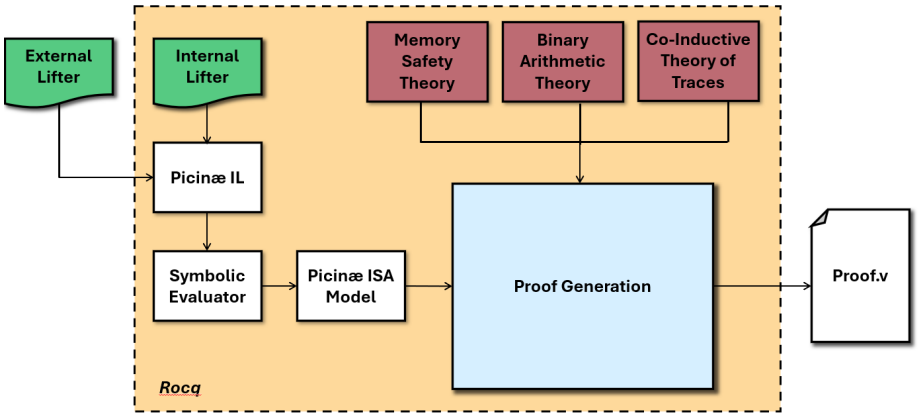


Fig. 1: Picinæ system

virtual address, yielding a mapping from each address to each instruction's IL representation. Recovering and verifying control-flows from this raw information is the province of proofs. Formally verifying the lifting process is the subject of prior works [26, 27]; our focus in this work is on verifying properties of lifted codes, and properties of binary rewriters that produce code as bytes.

Picinæ IL's semantics formally define machine states σ and *traces* τ of machine states, facilitating theorems that express code safety, security, and functional correctness as properties of states and traces. For example, theorems can assert that if $\tau = \sigma \ldots \sigma'$ is any reachable trace of the target program, and if σ satisfies a specified pre-condition, then σ' satisfies a specified post-condition. Theorem specifications can span all operators in Rocq's calculus of inductive constructors (CiC), and proofs can leverage any of Rocq's proof-construction languages, such as Ltac.

To support the task of constructing proofs, Picinæ provides a rich theory of supporting theorems and proof tactic automation libraries. This includes a fully machine-verified IL symbolic interpreter, binary arithmetic auto-simplification engine (for reducing expressions commonly arising from symbolic interpretation of binary code), inductive and co-inductive tactic libraries (for reasoning about loops and functions), temporal logic over traces, memory safety and pointer arithmetic theory, and architectural specializations of these for a variety of ISAs, including Intel x86/64, ARM7/8, and RISC-V. All support libraries remain strictly untrusted; they are machine-verified by the Rocq core.

Inductive and co-inductive tactics facilitate a standard proof strategy that organizes unstructured control-flow paths through target binaries as inductive propositions. Proofs declare *invariants* (machine state properties) at function entry points, loop entry points, and join-points, from which Picinæ auto-generates proof goals that verify each invariant-bounded control-flow path (and prove that no other control-flow paths are reachable). Solving all such goals creates a

machine-checked proof that all realizable traces satisfy the theorem. Co-induction affords partial correctness verification, and induction affords total correctness.

Picinæ's IL models single-threaded, uninterrupted execution without superscalar or speculative execution; however, it includes non-deterministic expressions to model unpredictable or implementation-defined cpu side-effects. Non-deterministic expressions become universally quantified proof variables in proofs, requiring proofs to show that theorems hold irrespective of any non-deterministic decisions made by the microarchitecture, such as scheduling.

2.1 Verification Example

[KWH comment: This subsection feels too far afield of our stated focus of verifying rewriters. I think we need to replace this with an example that verifies a very simple rewriter. Idea: What if we instead verify some very simple concrete ARM7 code (maybe strcmp for ARM7?), and then verify a Gallina algorithm that hotpatches it (i.e., overwrites the leading bytes with a jump instruction to a new instruction that replicates the overwritten instruction and jumps back)? This would be a very small but realistic (real ISA, real rewriting idiom) example that's more pursuant to our claimed novelty of verifying underconstrained-code properties.]

Figures 2 and 3 illustrate the workflow by verifying functional correctness of a binary implementation of strcmp from the gnu C standard libraries for Intel x86. Figure 2 shows the addressed assembly code on the left, and a suitable invariant set on the right. The invariant at `0x80` is a precondition that identifies the function entry point but makes no other assumptions. The loop invariant at `0x88` asserts that registers `ecx` and `edx` point to corresponding bytes of input strings s_1 and s_2, and that the string prefixes before those pointers are equal. The invariant at `0x96` and `0xa4` is the postcondition, asserting that the return value's sign equals the sign of a byte comparison between the first differing bytes in the two strings, or zero if there are no differing bytes.

The proof script in Figure 3 verifies that the machine state satisfies all declared invariants (including the postcondition) whenever they are reached, and no flows from the function entry point escape without first reaching the postcondition. Line 2 begins by applying Picinæ's main co-induction theorem, splitting the partial correctness proof into a base case and an inductive case. The base case is trivialized by locating the first invariant at the entry point (auto-solved by line 3), so the remainder of the proof consists of verifying the flow from each non-postcondition invariant to the next. Lines 4–5 set up these proof cases automatically. The flow from entry point to loop invariant is verified by line 6, the flow for a loop iteration is verified by line 9, and the two flows that exit the loop are verified by lines 10 and 11.

Throughout the proof, Picinæ's `step` tactic performs verified symbolic interpretation of each machine instruction, inferring its effect upon the machine state, and its `psimpl` tactic performs verified auto-simplification of binary arithmetic expressions in the resulting symbolic state. Both tactics auto-generate machine-

checked proofs of their results, and Rocq's native linear integer arithmetic solver `lia` auto-solves generated goals.

```
0x80: movl 0x4(%esp), %ecx
0x84: movl 0x8(%esp), %edx
0x88: movb (%ecx), %al
0x8a: cmpb (%edx), %al
0x8c: jne 0x9
0x8e: incl %ecx
0x8f: incl %edx
0x90: testb %al, %al
0x92: jne -0xc
0x94: xorl %eax, %eax
0x96: retl
0x97: movl $0x1, %eax
0x9c: movl $0xffffffff, %ecx
0xa1: cmovbl %ecx, %eax
0xa4: retl
```

Addr	Invariant
$0x80$	$\top$
$0x88$	$\exists k, \texttt{ecx} = s_1 + k \wedge \texttt{edx} = s_2 + k \wedge \texttt{mem}_{s_1}[0..k] = \texttt{mem}_{s_2}[0..k]$
$0x96$, $0xa4$	$\exists k, \texttt{mem}_{s_1}[0..k] = \texttt{mem}_{s_2}[0..k] \wedge (eax = 0 \rightarrow \texttt{mem}[s_1 + k] = 0) \wedge ((\texttt{eax} \mathrel{?=} 0) = (\texttt{mem}[s_1 + k] \mathrel{?=} \texttt{mem}[s_2 + k]))$

Fig. 2: Assembly (left) and invariant set (right) for i386 strcmp.
TODO: format the assembly code.

```
Proof.
  apply prove_invs. (* begin co-induction *)
  + easy. (* base case: ⊤ *)
  + apply strcmp_preserves_memory.
    destruct_inv 32 PRE. (* inductive case *)
    - step. step. exists 0. psimpl. repeat (lia || split). (* entry -> loop *)
    - destruct PRE as (k & ECX & EDX & SEQ).
      repeat step.
      * exists (k+1). ... (* loop -> loop *)
      * ... (* loop -> exit 0x96 *)
      * ... (* loop -> exit 0xa4 *)
Qed.
```

Fig. 3: Proof of partial correctness for i386 strcmp.

3 System Implementation

3.1 Verification Infrastructure

Figures 4 and 5 show Picinæ IL's syntax and semantics, respectively. Programs are functions that map a cpu state σ and virtual address a to an IL *statement* s that models the semantics of the machine instruction located at a and its size sz in bytes. In von Neumann architectures, p derives its output from the memory state in σ by decoding the bytes at a into an assembly instruction whose semantics are lifted to s. This feature is critical for verifying a binary rewriter,

Expressions	$e ::= v \mid n_{\boxed{w}} \mid e_1[e_2]_{\boxed{w}} \mid e_1[e_2 := e_3]_{\boxed{w}} \mid e_1 \circledcirc e_2 \mid (e : w) \mid \mathsf{let}\ v := e_1\ \mathsf{in}\ e_2 \mid *_{\boxed{w}}$
Statements	$s ::= \cdot \mid v := e \mid \texttt{jmp}\ e \mid \texttt{exn}\ n \mid s_1; s_2 \mid e\ ?\ s_1 : s_2 \mid s^e$
Contexts & Stores	$c, \sigma : \Sigma := v \rightharpoonup \mathbb{N}$
Exits	$\chi ::= \ \downarrow$ (fallthru) $\mid a$ (destination address) $\mid \uparrow i$ (hardware exception)
Programs	$p : \Sigma \to \mathbb{N} \rightharpoonup s$
Traces	$\tau ::= \langle \chi, \sigma \rangle \mid \langle \chi, \sigma \rangle :: \tau$

Fig. 4: Picinæ IL syntax

since it makes the ISA's instruction decoding logic available to proofs. Proofs may thereby verify that a rewriter's bit-manipulation logic correctly detects and/or generates ISA instructions with particular semantic properties.

Statements s affect the cpu state σ via no-operation ($\cdot$), destructive updates ($:=$) of machine state elements v (e.g., registers and memory), jumps (`jmp`), hardware exceptions (`exn`), sequences (;), conditionals (?), and finite repetitions (s^e). Finite repetitions model individual instructions whose internal semantics contain a bounded micro-loop (e.g., the `rep` instruction prefix on Intel ISAs). Expressions include state element reads (v), unsigned binary numbers n of bit width w ($n_{\boxed{w}}$), w-byte reads of memory e_1 from address e_2 ($e_1[e_2]_{\boxed{w}}$), w-byte memory-writes ($e_1[e_2 := e_3]_{\boxed{w}}$), binary modular arithmetic operations ($\circledcirc$), bit width casts ($e : w$), let-bindings, and non-determinism ($*_{\boxed{w}}$). Non-determinism models side-effects that the ISA specifies are unpredictable or undefined. For example, the Intel x86-64 ISA specifies that `xor` instructions have undefined effects upon the cpu's 1-bit arithmetic flag, which the lifter models as $\texttt{AF} := *_{\boxed{1}}$.

Statements and expressions evaluate within an ISA *context* c that specifies the bit width of each cpu state element. States may also include additional *abstract* elements that are not part of a real cpu's state (e.g., for temporary values). Virtual memory of address width w is modeled as a single state element of bit width $2^{2^{w+3}}$. A static semantics (not shown) ensures that each non-abstract assignment $v := e$ preserves the ISA-defined bit width of v. IL from external lifters is type-checked by a machine-verified type-checker, and internal lifters are machine-verified to be universally well-typed. This relieves code verification proofs from re-proving fundamental properties of the ISA.

Programs p exhibit *traces* τ, which list cpu states paired with *exits* χ (*viz.*, the current virtual address a or hardware exception number $\uparrow i$). ISA function exitof defines the next virtual address reached by machine instructions that fall through (i.e., when $\chi = \downarrow$). The program-trace relation $p \rightsquigarrow \tau$ is non-deterministic: For any given p, relation $p \rightsquigarrow \tau$ is satisfied by all traces τ that constitute valid execution sequences admitted by the ISA specification. Partial correctness theorems therefore typically have the form $\forall \tau, (c \vdash p \rightsquigarrow \tau) \to P(\tau)$, which asserts that every trace τ realizable by program p in ISA context c satisfies proposition P.

Partial correctness proofs about concrete binary code are structured using *invariant sets*—partial functions from virtual addresses to machine state prop-

$$\frac{}{c,\sigma \vdash v \Downarrow \sigma(v)_{\boxed{c(v)}}}(\text{EVar}) \qquad \frac{}{c,\sigma \vdash n_{\boxed{w}} \Downarrow n_{\boxed{w}}}(\text{EWord}) \qquad \frac{c,\sigma \vdash e_1 \Downarrow n_{\boxed{w}} \quad c,\sigma \vdash e_2 \Downarrow n'_{\boxed{w}}}{c,\sigma \vdash e_1 \circledcirc e_2 \Downarrow (n \circledcirc_w n')_{\boxed{w}}}(\text{EBinOp})$$

$$\frac{c,\sigma \vdash e_1 \Downarrow m_{\boxed{8(1 \ll w_0)}} \quad c,\sigma \vdash e_2 \Downarrow a_{\boxed{8w_0}}}{c,\sigma \vdash e_1[e_2]_w \Downarrow m[8a..8(a{+}w)]_{\boxed{8w}}}(\text{ELoad}) \qquad \frac{c,\sigma \vdash e_1 \Downarrow m_{\boxed{8(1 \ll w_0)}} \quad c,\sigma \vdash e_2 \Downarrow a_{\boxed{8w_0}} \quad c,\sigma \vdash e_3 \Downarrow n_{\boxed{8w}}}{c,\sigma \vdash e_1[e_2 \coloneqq_w e_3] \Downarrow m[8a \coloneqq_w n]_{\boxed{8(1 \ll w_0)}}}(\text{EStore})$$

$$\frac{c,\sigma \vdash e \Downarrow n_{\boxed{w_1}}}{c,\sigma \vdash (e : w) \Downarrow (n \bmod 2^w)_{\boxed{w}}}(\text{ECast}) \qquad \frac{c,\sigma \vdash e_1 \Downarrow n_{\boxed{w_1}} \quad c[v{\coloneqq}w_1],\sigma[v{\coloneqq}n] \vdash e_2 \Downarrow n_{\boxed{w_2}}}{c,\sigma \vdash \mathsf{let}\ v \coloneqq e_1\ \mathsf{in}\ e_2 \Downarrow n_{\boxed{w_2}}}(\text{ELet}) \qquad \frac{0 \le n < 2^w}{c,\sigma \vdash *_w \Downarrow n_{\boxed{w}}}(\text{EUnk})$$

$$\frac{}{c,\sigma \vdash \cdot \Downarrow_{\downarrow} c,\sigma}(\text{XNop}) \qquad \frac{c,\sigma \vdash e \Downarrow n_{\boxed{w}}}{c,\sigma \vdash v \coloneqq e \Downarrow_{\downarrow} c[v \coloneqq w],\sigma[v \coloneqq n]}(\text{XMove}) \qquad \frac{c,\sigma \vdash e \Downarrow a_{\boxed{w}}}{c,\sigma \vdash \mathtt{jmp}\ e \Downarrow_a c,\sigma}(\text{XJmp})$$

$$\frac{}{c,\sigma \vdash \mathtt{exn}\ i \Downarrow_{\uparrow i} c,\sigma}(\text{XExn}) \qquad \frac{c,\sigma \vdash s_1 \Downarrow_\chi c',\sigma' \quad \chi \neq \downarrow}{c,\sigma \vdash s_1; s_2 \Downarrow_\chi c',\sigma'}(\text{XSeq1}) \qquad \frac{c,\sigma \vdash s_1 \Downarrow_{\downarrow} c_2,\sigma_2 \quad c_2,\sigma_2 \vdash q_2 \Downarrow_\chi c',\sigma'}{c,\sigma \vdash s_1; s_2 \Downarrow_\chi c',\sigma'}(\text{XSeq2})$$

$$\frac{c,\sigma \vdash e \Downarrow n_w \quad c,\sigma \vdash s_{n\neq 0} \Downarrow_\chi c',\sigma'}{c,\sigma \vdash e\ ?\ s_{\mathsf{T}} : s_{\mathsf{F}} \Downarrow_\chi c',\sigma'}(\text{XIf}) \qquad \frac{c,\sigma \vdash e \Downarrow n_w \quad c,\sigma \vdash s;\overset{n}{\cdots};s \Downarrow_\chi c',\sigma'}{c,\sigma \vdash s^e \Downarrow_\chi c',\sigma'}(\text{XRep})$$

$$\frac{}{c \vdash p \rightsquigarrow \langle \chi,\sigma\rangle}(\text{PStart}) \qquad \frac{c,\sigma \vdash p\sigma a \Downarrow_\chi c',\sigma' \quad c \vdash p \rightsquigarrow \langle a,\sigma\rangle :: \tau}{c \vdash p \rightsquigarrow \langle \mathsf{exitof}(\chi,a,p\sigma),\sigma'\rangle :: \langle a,\sigma\rangle :: \tau}(\text{PStep})$$

Fig. 5: Picinæ semantics: expression evaluation (E∗), statement execution (X∗), program-trace modeling (P∗).

erties. Code verification is a co-induction with two cases: (1) all traces through program p satisfy the first invariant they reach (the base case), and (2) every trace beginning at a satisfied invariant satisfies any next invariant it reaches. Invariant sets automatically place fallacious invariants at all virtual addresses outside the target code fragment, guaranteeing that all such proofs provide full coverage (i.e., no reachable control-flow paths escape).

Formally, a program p with exit points $xp \subset \mathbb{N}$ satisfies invariant set $Invs : a \rightharpoonup \Sigma \to \mathsf{Prop}$ if all realizable traces satisfy all reached invariants before exiting:

$$NoExit\ \tau \coloneqq \forall \langle a,\sigma\rangle \in \tau,\ a \notin xp \tag{1}$$

$$SatAll\ \tau \coloneqq \forall a,\sigma,P,\ (c \vdash p \rightsquigarrow \langle a,\sigma\rangle{::}\tau) \to NoExit\ \tau \to (Invs\ a = P)\,(\sigma) \tag{2}$$

These definitions promote uniform theorem statements that assert standard assumptions about the initial machine state of trace τ (e.g., that it begins at the function entry point with a well-formed stack frame) and conclude $SatAll\ \tau$. Proofs make progress by leveraging antecedent $c \vdash p \rightsquigarrow \langle a,\sigma\rangle{::}\tau$ of $SatAll$ to infer that either (a) $a \in \mathrm{dom}(Invs)$ is an invariant, which becomes the current proof goal; or (b) $a \notin \mathrm{dom}(Invs)$ must take at least one computational step before reaching any next invariant.

Picinæ's `step` tactic automates the latter case. It launches a machine-verified symbolic interpreter that functionally infers the next symbolic machine state ac-

cording to the IL semantics in Figure 5. This acts as an abstract machine code debugger that executes one machine instruction and generates a machine-checkable proof that its results are correct. When the instruction is finite-branching, multiple next states are generated as multiple proof goals. When it is non-deterministic or infinite-branching, the new states contain fresh, universally quantified proof variables that reflect all non-deterministic choices. Proofs must therefore confirm that theorems hold for all possible branches and non-deterministic effects. Invariant sets curtail state space explosion by coalescing all paths at invariant points to common invariant-defined proof goals.

3.2 Binary Rewriter Verification

Binary rewriters read byte sequences, interpret them as machine code, and write new byte sequences intended to be interpreted as machine code by the ISA. Our work focuses on verifying a rewriter implemented in a high-level source language (Gallina) amenable to top-down FM, but we formulate and proof a soundness property that concerns the full universe of binary codes that it may read and write once deployed, including all possible execution traces of those codes.

TODO: We need to get back to binary rewriters here, and there's a lot to say. Some/all of the "Correctness of Binary Rewriters" section probably goes here. But more than what's currently there needs to be written. Some things from "Proof Architecture" subsection should also go here. Basically we want to give formal definitions of the main theorems and proof strategy for binary rewriter verification in general, without going into the messy details of our specific proof for our specific rewriter. A summary of our particular proof should go later (in "evaluation").

3.3 Possibly Too Much for This Paper

The theory library in part defines and proves properties of PIL, including well-typedness of statements and their confluence. It comprises number-theoretic results, abstractions and theory of binary-level data structures such as strings and linked lists, and logical properties of Picinæ's representations and abstractions that have been useful in binary-level formal verification. TODO: What else do we want to say about theory?

One of the challenges faced with deductive verification of binary code is that expressions quickly grow too large to read and become cumbersome to manipulate. Picinæ's simplifier *psimpl* addresses this issue for expressions yielded by the symbolic interpreter, including store updates and reads, modular and bitwise arithmetic, and bit extraction. Psimpl is a reflexive tactic, effecting simplifications via computation certified with a proof of correctness. The simplifier is a three-stage pipeline that overcomes both theoretical challenges of rewriting logics and the implementation idiosyncrasies of the Rocq proof assistant. The first stage overcomes the computation engine's aggressive unfolding of definitions, even ones marked as opaque. Psimpl first turns expressions into simplifier abstract syntax trees (SAST's), a deep embedding of Rocq's terms into a data

structure whose semantics depend on a mapping from inductive symbols to the Rocq functions they signify. This transforms the expression into one whose computation content is abstracted into a semantics, so vm_compute only evaluates the function reducing the SAST, it never sees the computation functions it would reduce, such as add and getmem.

The second stage simplifies the SAST using a postorder traversal. The most important reduction is the elimination of modulo operations. This depends on propogating upper and lower bounds of expressions during a traversal, simplifying modulo operations when the upper bound of the expression is less than the modulus. Other simplifications include eliminating redundant update operations on stores, interpreting store lookups, and reducing arithmetic expressions, e.g., reducing $x+y-y'$ to x when y and y' are provably equal. After the simplifications the third stage transforms the SAST back into a Picinæ expression.

Picinæ facilitates modular reasoning about programs by supporting call-return semantics. Once proved,the partial correctness proof of a callee can be applied inside of the proof of a caller to skip re-proving the callee providing sufficient conditions are met. First, the callee invariants must imply any collocated caller invariants. Second, the callee must not exit other than by reaching the return site, i.e., it must not raise a hardware exception or jump to a non-code address. Third, the callee's invariants must be functions of the cpu state, i.e. they must be *prefix-closed* [28]. Picinæ provides infrastructure for defining invariant sets and exits such that these conditions are guaranteed to be met for non-overlapping code fragments.

Picinæ treats indirect calls and jumps like all other instructions. Specifically, as a `jmp` e statement, with e a non-constant expression.

but the address being an expression rather than a constant complicates how the next instruction to execute is computed. If the indirect call is to a function with an axiomatized partial correctness property, e.g., a function pointer to a comparator, then the function-call machinery can dispatch the function without the prover needing to define code in its section of memory. In other cases the prover must specify what bytes are in memory or what that memory address decodes to. This, in fact, is the basis for reasoning about binary transformations over arbitrary binary code.

4 Correctness of CFI Rewriters

The correctness specification of a CFI rewriter describes the rewriter's ability to enforce a control-flow policy for the programs it produces. A rewritten program abides by its policy if the control flow transfer from each IRM either goes to the abort destination or corresponds to a policy-permitted transfer in the original program. A proof that a CFI rewriter conforms to this specification must therefore reason about all execution steps that may be taken in the produced program.

CFI rewriters replace instructions with *inline reference monitors* (IRMs), instruction sequences that constrain control-flow to the policy. We can partition

a processor's execution history into IRM block executions, enabling an inductive proof structure for the correctness of the rewriter. To show that programs produced by the rewriter always abide by the policy, it suffices to show that all IRMs a rewriter produces abide by the policy.

Each rewritten instruction falls into one of four categories, leading to another tier of case analysis for a rewriter's correctness proof. It is either

1. a safe instruction,
2. a static jump (i.e., destination is known at compile time),
3. a dynamic jump (e.g., function pointers, jump tables, returns), or
4. an invalid instruction.

Case 1 requires no rewriting. Case 2 requires replacing the original jump target with a new location, as inserted instructions will relocate most code. Case 3 requires augmenting the jump with a mechanism to ensure that the target is in the policy (e.g. a lookup table). Case 4 requires replacing invalid instructions with jumps to an abort routine.

Thus, to prove that a CFI rewriter is correct, it suffices to show that all code generated by the rewriter in each of the above four cases never leaves the policy, or leaves the policy and abandons safety guarantees (e.g., syscall). We show that our CFI rewriter implementation is correct by a proof that follows this inductive structure.

5 A Verified Binary CFI Rewriter

We have implemented a CFI rewriter in Gallina using the aforementioned case-analytical structure. The rewriter accepts an ARMv7 binary program and a control-flow policy as input and produces a rewritten binary with IRMs that enforce the policy. Furthermore, we have proved the implementation to be correct using the proof structure outlined above. The result is an extractable OCaml program with machine-checked correctness guarantees.

5.1 Rewriter Architecture

Our rewriter operates by analyzing each instruction in the input program and dispatching to specialized rewriting functions based on the four cases identified earlier. The overall architecture follows a compositional design where each component handles one category of instruction sequences and produces provably-safe inline reference monitors.

Safe Instructions are those that do not affect control flow: arithmetic operations, memory loads and stores to non-executable regions, and other data processing instructions. These instructions require no rewriting for CFI enforcement, as they cannot violate the control-flow policy. However, they must still be relocated to account for the insertion of IRMs elsewhere in the program. Our rewriter preserves these instructions unchanged but updates any PC-relative addressing to reflect their new locations.

Static Branches (i.e., direct jumps and calls with compile-time constant targets) are inherently policy-compliant if their target is valid according to the CFI policy. The challenge is that IRM insertion causes code relocation, invalidating original branch targets. Our rewriter handles static branches by computing new target addresses that account for the cumulative size of all IRMs inserted before the target location. The rewritten instruction uses the updated immediate offset to reach the intended destination.

For conditional branches, we preserve the original condition code while updating only the target offset. This ensures that the dynamic control flow behavior matches the original program while satisfying the CFI policy.

Dynamic Branches (e.g., indirect jumps, returns, and jump tables) pose the primary challenge for CFI enforcement, as their targets are computed at runtime. Our rewriter instruments these instructions with table-lookup IRMs that validate destinations before transferring control.

Invalid instructions in the original binary could potentially be exploited to bypass CFI checks. Our rewriter replaces any invalid or unpredictable instruction sequences with unconditional branches to a designated abort routine. The abort routine terminates execution safely, preventing any policy violations.

Hash-Based Table Lookup. Unlike traditional CFI implementations that use linear arrays indexed directly by target address, our approach uses a compact hash table indexed by a hash of the target address. For each dynamic branch site, the rewriter:

1. Computes an optimal hash function of the form $H(x) = (x \ll sl) \gg sr$, where sl and sr are shift parameters chosen to prevent collisions among the policy-permitted destinations for this branch site.
2. Generates a lookup table where $table[H(d)] = d'$ and $table[H(d')] = d'$ for each permitted destination d (mapped to its new location d').
3. Inserts validation code that computes $H(target)$, looks up the result in the table, and transfers control to the retrieved address, which is either permitted by the policy or the abort address.

5.2 Proof Architecture

$$\begin{aligned} \forall\ & pol, code, irms, tables, s_0, t, \\ & \texttt{cfi_rw}(pol, code) = \texttt{Some}(code', tables) \wedge \\ & valid(abort_index) \wedge \\ & store_invs(s_0) \implies \\ & satisfies_all(code', cfi_invs, cfi_exits, t) \end{aligned} \tag{3}$$

Equation 3 presents our main safety theorem, which states that any execution trace starting at the entrypoint of code produced by `cfi_rw` will satisfy all CFI invariants. Each invariant states that at the beginning of each IRM block, either

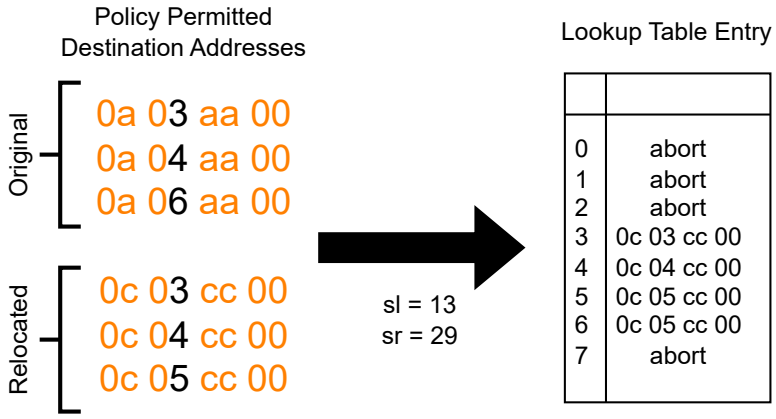


Fig. 6: Example Hash Table Entry

1. control will always reach a CFI invariant point and satisfy the safety guarantee placed there,
2. control will leave the bounds of the rewritten code (e.g. to interface with the operating system), at which point no security guarantee is provided, or
3. a hardware exception is raised.

5.3 Supporting Safety Lemmas

The main safety theorem is proved through a hierarchy of supporting lemmas that establish safety properties for each IRM category. These lemmas follow a common structure: they take the rewriter output for a specific instruction, assume the initial store is well-formed, and prove that execution satisfies the permitted step relation and reaches the next CFI invariant point. Well-formed stores must have the IRM code and lookup tables at the correct location in memory, and these regions must be marked non-writable.

Static Jumps and Safe Instructions. For safe instructions, we prove a *fallthrough lemma* establishing that execution proceeds to the next instruction while preserving all invariants. The key insight is that safe instructions cannot modify control flow.

Safe instructions are characterized by (1) never leading to an exit point, and (2) maintaining critical processor flags such as endianness and instruction set. Individual lemmas establish that common instructions (e.g., loads, stores, arithmetic operations, and bit manipulations) satisfy this predicate by analyzing their PIL semantics.

For static branches, the proof must relate the original branch target to its relocated counterpart. We define address mapping functions $i2i'$ and $i'2i$ that convert between original and relocated addresses. The key lemma shows that

executing a rewritten static branch transfers control to an address a' such that either $a' = a_{\text{abort}}$, or the original block index $i'2i(\frac{a'}{4})$ is policy-permitted from the source block.

An important subtlety is handling conditional branches. The rewriter wraps conditional IRMs with a branch that skips the IRM body when the condition is false. The proof uses case analysis on the condition evaluation: if false, execution falls through to the next block; if true, the IRM executes and the nested lemma applies.

Dynamic Jumps. Dynamic jump safety requires a multi-stage proof corresponding to the hash table construction and lookup process.

We first prove that the hash parameter search produces collision-free functions. The rewriter searches for shift parameters (sl, sr) such that applying $H(x) = (x \ll sl) \gg sr$ to all policy-permitted destinations produces distinct values, with at most one collision between an original destination d and its relocated counterpart d'. This collision tolerance is essential because runtime register values might contain either the original or relocated address depending on execution history.

Given valid hash parameters, we prove that the generated table maps each hash value to a safe destination. The proof proceeds by induction on the policy's destination list, showing that each table entry is either the abort address or a relocated destination corresponding to a policy-permitted original address. This requires establishing a predicate for *safe exits*: an address a is safe from source block i if either $a = a_{\text{abort}}$, or there exists an original destination d such that $i2i'(d) = a$ and $d \in pol(i)$.

The table lookup sequence consists of hash computation, address scaling, table base addition, and load. The composition of these steps shows that the final loaded jump target satisfies the safe exit predicate.

Invalid Instructions. Invalid or unpredictable instructions are replaced with unconditional branches to the abort routine. The proof is straightforward: we symbolically execute the generated instruction and show that it unconditionally transfers control to a_{abort}, which trivially satisfies the permitted step relation.

All potential invalid instructions are provably caught by this verified replacement. The rewriter explicitly handles all known instructions, and replaces all unknown instructions with jumps to the abort routine.

Proof Composition The final safety theorem composes these category-specific lemmas through a master case analysis lemma that examines the rewriter's output for each instruction. Given the rewriter's output for instruction at index i, the lemma:

1. Determines which category the instruction falls into based on the rewriter's behavior

2. Retrieves the appropriate category lemma
3. Applies the lemma to establish that execution from address i' satisfies permitted steps and reaches the next invariant point at $i' + |\text{irm}|$

This structure enables modular reasoning: each category's safety is proved independently, and the composition is purely structural case analysis. The proof of the main safety theorem then proceeds by induction on the execution trace, applying this per-instruction lemma at each IRM block boundary.

5.4 Supporting Formalizations

5.5 Construction of Secure Binary

TODO: Do we need to discuss the python script that reconstructs the binaries?

5.6 Symbolic Execution of Quantified Programs

Proving some instruction blocks are safe requires reasoning about their destination address and its relationship to the table returned by the rewriter. The destination address is computed from the table through a sequence of instructions, and the proof requires reasoning about those instructions despite the absence of a concrete program to analyze. It turns out that for symbolic execution, it is sufficient to know that the block is in the same memory address that the trace is executing from, exactly our assumptions.

6 Evaluation

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

7 Related Work

CFI enforcement is a very active field of research with many new publications each year, each supporting the correctness of their work using one or more of three approaches [29]: theoretical verification, experimental verification or quantitative analysis. Theoretical verification comprises informal analyses of the CFI solution (e.g., `SAFEDISPATCH` [12]), experimental verification includes applying the CFI technique to vulnerability test-suites (e.g., RIPE [30] or RecIPE [31]) or other software with known vulnerabilities (e.g., CVEs), and quantitative analysis involves measuring a static property of programs before and after treatment, such as number of code gadgets or average indirect target reduction (AIR) [32]. In contrast, we evaluated the security of our CFI *implementation* by *formally* verifying it, thus eliminating the possibility of errors in implementation, which account for 5 of 7 common causes of error in CFI [29].

Recent work has started to explore the problem of machine-verifying underconstrained-code properties by examining relational properties of *concrete* programs with statically known code [33]. TODO: Follow this with an analysis of the form, "This accomplishes (good things) but cannot achieve (our achievements)."

TODO: What other aspects of CFI do we want to cover? Do we want to go into the strategies employed by other binary rewriters? Do we want to discuss source-dependent solutions at all?

Formal verification of low-level code has received increasing attention recently. Most projects have created general purpose binary analysis frameworks, but some have tailored solutions to specific problems, such as certified decompilation [26] and binary pointer analysis [34]. Islaris [22] reasons about concrete program traces using the Isla symbolic interpreter to create trusted traces from the target architecture's Sail specification. IsaBIL [21] translates BAP [25] decompilations into its Isabelle/HOL embedding to reason about concrete program executions. Most recently, a team published a relational Hoare logic in HOL Light and used it to prove program equivalence and correctness [33]. Of these, only the last is capable of quantifying over programs, as we have done, because it uses a decode function to interpret bytes from memory.

8 Conclusion

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

References

[1] Xavier Leroy. "Formal verification of a realistic compiler". In: *Commun. ACM* 52.7 (July 2009), pp. 107–115. ISSN: 0001-0782. DOI: 10.1145/1538788.1538814. URL: https://doi.org/10.1145/1538788.1538814.

[2] Gerwin Klein et al. "seL4: formal verification of an OS kernel". In: *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*. SOSP '09. Big Sky, Montana, USA: Association for Computing Machinery, 2009, pp. 207–220. ISBN: 9781605587523. DOI: 10.1145/1629575.1629596. URL: https://doi.org/10.1145/1629575.1629596.

[3] Ramana Kumar et al. “CakeML: a verified implementation of ML”. In: *SIGPLAN Not.* 49.1 (Jan. 2014), pp. 179–191. ISSN: 0362-1340. DOI: 10.1145/2578855.2535841. URL: https://doi.org/10.1145/2578855.2535841.

[4] The Rocq Development Team. *The Rocq Prover*. Mar. 2025. URL: https://rocq-prover.org.

[5] Timothy Bourke et al. “A formally verified compiler for Lustre”. In: *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*. PLDI 2017. Barcelona, Spain: Association for Computing Machinery, 2017, pp. 586–601. ISBN: 9781450349888. DOI: 10.1145/3062341.3062358. URL: https://doi.org/10.1145/3062341.3062358.

[6] José Bacelar Almeida et al. “Certified Compilation for Cryptography: Extended x86 Instructions and Constant-Time Verification”. In: *Progress in Cryptology – INDOCRYPT 2020*. Ed. by Karthikeyan Bhargavan, Elisabeth Oswald, and Manoj Prabhakaran. Cham: Springer International Publishing, 2020, pp. 107–127. ISBN: 978-3-030-65277-7.

[7] Wolfgang Meier et al. “CertiCoq-Wasm: A Verified WebAssembly Backend for CertiCoq”. In: *Proceedings of the 14th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP ’25. Denver, CO, USA: Association for Computing Machinery, 2025, pp. 127–139. ISBN: 9798400713477. DOI: 10.1145/3703595.3705879. URL: https://doi.org/10.1145/3703595.3705879.

[8] Jérémie Koenig and Zhong Shao. “CompCertO: compiling certified open C components”. In: *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. PLDI 2021. Virtual, Canada: Association for Computing Machinery, 2021, pp. 1095–1109. ISBN: 9781450383912. DOI: 10.1145/3453483.3454097. URL: https://doi.org/10.1145/3453483.3454097.

[9] Oghenevwogaga Ebresafe et al. “Certified Compilers à la Carte”. In: *Proc. ACM Program. Lang.* 9.PLDI (June 2025). DOI: 10.1145/3729261. URL: https://doi.org/10.1145/3729261.

[10] George C. Necula and Peter Lee. “The design and implementation of a certifying compiler”. In: *SIGPLAN Not.* 33.5 (May 1998), pp. 333–344. ISSN: 0362-1340. DOI: 10.1145/277652.277752. URL: https://doi.org/10.1145/277652.277752.

[11] Luís Cruz-Filipe, Lovro Lugović, and Fabrizio Montesi. “Certified Compilation of Choreographies with hacc”. In: *Formal Techniques for Distributed Objects, Components, and Systems*. Ed. by Marieke Huisman and António Ravara. Cham: Springer Nature Switzerland, 2023, pp. 29–36. ISBN: 978-3-031-35355-0.

[12] Dongseok Jang, Zachary Tatlock, and Sorin Lerner. “SafeDispatch: Securing C++ Virtual Calls from Memory Corruption Attacks”. In: *21st Annual Network and Distributed System Security Symposium, NDSS 2014, San Diego, California, USA, February 23-26, 2014*. The Internet Society, 2014.

URL: https://www.ndss-symposium.org/ndss2014/safedispatch-securing-c-virtual-calls-memory-corruption-attacks.

[13] Victor Van Der Veen et al. "A tough call: Mitigating advanced code-reuse attacks at the binary level". In: *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE. 2016, pp. 934–953.

[14] Wenhao Wang, Xiaoyang Xu, and Kevin W Hamlen. "Object flow integrity". In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2017, pp. 1909–1924.

[15] Alexander J Gaidis et al. "Fineibt: Fine-grain control-flow enforcement with indirect branch tracking". In: *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*. 2023, pp. 527–546.

[16] Erick Bauman et al. "Renewable Just-In-Time Control-Flow Integrity". In: *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*. 2023, pp. 580–594.

[17] Hao Xiang et al. "Boosting Practical Control-Flow Integrity with Complete Field Sensitivity and Origin Awareness". In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2024, pp. 4524–4538.

[18] Gianluca Roascio, Nicolò Maunero, and Gabriele Costa. "Control-Flow Integrity for Resource-Constrained Devices". In: *Computers & Security* (2025), p. 104730.

[19] Jennifer Miller et al. "System register hijacking: Compromising kernel integrity by turning system registers against the system". In: *34th USENIX Security Symposium (USENIX Security 25)*. 2025, pp. 7427–7446.

[20] Andreas Lindner, Roberto Guanciale, and Roberto Metere. "TrABin: Trustworthy analyses of binaries". In: *Science of Computer Programming* 174 (2019), pp. 72–89. ISSN: 0167-6423. DOI: https://doi.org/10.1016/j.scico.2019.01.001. URL: https://www.sciencedirect.com/science/article/pii/S0167642318301667.

[21] Matt Griffin, Brijesh Dongol, and Azalea Raad. "IsaBIL: A Framework for Verifying (In)correctness of Binaries in Isabelle/HOL". In: *39th European Conference on Object-Oriented Programming, ECOOP 2025, June 30 to July 2, 2025, Bergen, Norway*. Ed. by Jonathan Aldrich and Alexandra Silva. Vol. 333. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025, 14:1–14:30. DOI: 10.4230/LIPICS.ECOOP.2025.14. URL: https://doi.org/10.4230/LIPIcs.ECOOP.2025.14.

[22] Michael Sammler et al. "Islaris: verification of machine code against authoritative ISA semantics". In: *PLDI '22: 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, San Diego, CA, USA, June 13 - 17, 2022*. Ed. by Ranjit Jhala and Isil Dillig. ACM, 2022, pp. 825–840. DOI: 10.1145/3519939.3523434. URL: https://doi.org/10.1145/3519939.3523434.

[23] Magnus O. Myreen, Michael J. C. Gordon, and Konrad Slind. "Machine-Code Verification for Multiple Architectures - An Application of Decom-

pilation into Logic". In: *Formal Methods in Computer-Aided Design, FMCAD 2008, Portland, Oregon, USA, 17-20 November 2008*. Ed. by Alessandro Cimatti and Robert B. Jones. IEEE, 2008, pp. 1–8. DOI: 10.1109/FMCAD.2008.ECP.24. URL: https://doi.org/10.1109/FMCAD.2008.ECP.24.

[24] Björn Bartels and Nils Jähnig. "Mechanized, Compositional Verification of Low-Level Code". In: *NASA Formal Methods*. Ed. by Julia M. Badger and Kristin Yvonne Rozier. Cham: Springer International Publishing, 2014, pp. 98–112. ISBN: 978-3-319-06200-6. URL: https://doi.org/10.1007/978-3-319-06200-6_8.

[25] David Brumley et al. "BAP: A Binary Analysis Platform". In: *Computer Aided Verification - 23rd International Conference, CAV 2011, Snowbird, UT, USA, July 14-20, 2011. Proceedings*. Ed. by Ganesh Gopalakrishnan and Shaz Qadeer. Vol. 6806. Lecture Notes in Computer Science. Springer, 2011, pp. 463–469. DOI: 10.1007/978-3-642-22110-1_37. URL: https://doi.org/10.1007/978-3-642-22110-1%5C_37.

[26] Freek Verbeek et al. "Formally Verified Lifting of C-compiled x86-64 Binaries". In: *Proceedings of the 43rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2022, pp. 934–949.

[27] Freek Verbeek, Nico Naus, and Binoy Ravindran. "Verifiably Correct Lifting of Position-Independent x86-64 Binaries to Symbolized Assembly". In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. CCS '24. Salt Lake City, UT, USA: Association for Computing Machinery, 2024, pp. 2786–2798. ISBN: 9798400706363. DOI: 10.1145/3658644.3690244. URL: https://doi.org/10.1145/3658644.3690244.

[28] Fred B. Schneider. "Enforceable security policies". In: *ACM Trans. Inf. Syst. Secur.* 3.1 (2000), pp. 30–50. DOI: 10.1145/353323.353382. URL: https://doi.org/10.1145/353323.353382.

[29] Yuan Li et al. "Finding Cracks in Shields: On the Security of Control Flow Integrity Mechanisms". In: *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*. Ed. by Jay Ligatti et al. ACM, 2020, pp. 1821–1835. DOI: 10.1145/3372297.3417867.

[30] John Wilander et al. "RIPE: runtime intrusion prevention evaluator". In: *Twenty-Seventh Annual Computer Security Applications Conference, ACSAC 2011, Orlando, FL, USA, 5-9 December 2011*. Ed. by Robert H'obbes' Zakon, John P. McDermott, and Michael E. Locasto. ACM, 2011, pp. 41–50. DOI: 10.1145/2076732.2076739. URL: https://doi.org/10.1145/2076732.2076739.

[31] Yuancheng Jiang et al. "RecIPE: Revisiting the Evaluation of Memory Error Defenses". In: *ASIA CCS '22: ACM Asia Conference on Computer and Communications Security, Nagasaki, Japan, 30 May 2022 - 3 June*

2022. Ed. by Yuji Suga et al. ACM, 2022, pp. 574–588. DOI: 10.1145/3488932.3524127. URL: https://doi.org/10.1145/3488932.3524127.

[32] Mingwei Zhang and R. Sekar. "Control Flow Integrity for COTS Binaries". In: *Proceedings of the 22th USENIX Security Symposium, Washington, DC, USA, August 14-16, 2013*. Ed. by Samuel T. King. USENIX Association, 2013, pp. 337–352. URL: https://www.usenix.org/conference/usenixsecurity13/technical-sessions/presentation/Zhang.

[33] Denis Mazzucato et al. "Relational Hoare Logic for Realistically Modelled Machine Code". In: *Computer Aided Verification - 37th International Conference, CAV 2025, Zagreb, Croatia, July 23-25, 2025, Proceedings, Part I*. Ed. by Ruzica Piskac and Zvonimir Rakamaric. Vol. 15931. Lecture Notes in Computer Science. Springer, 2025, pp. 389–413. DOI: 10.1007/978-3-031-98668-0_19. URL: https://doi.org/10.1007/978-3-031-98668-0_19.

[34] Freek Verbeek et al. *Formally Verified Binary-level Pointer Analysis*. 2025. arXiv: 2501.17766 [cs.SE]. URL: https://arxiv.org/abs/2501.17766.

[35] Chris Delikat Brian Knighton. *Ghidra - Journey from Classified NSA Tool to Open Source*. Aug. 2019.